

Calcolatori Elettronici

25 febbraio 2025

Teoria

Esercizio 1 (7 punti): Siano X e Y la penultima e l'ultima cifra della propria matricola. Convertire in formato IEEE 754 a 32 bit il valore $-5X.Y1$ mostrando tutti i passaggi—occorre *sostituire* le cifre della matricola a X e Y , non eseguire una moltiplicazione. Rappresentare il risultato anche in esadecimale.

Esercizio 2 (11 punti): Si progetti una macchina sequenziale sincrona in grado di determinare il vincitore di un set di tennis giocato al meglio di tre¹. La macchina riceve in ingresso, ad ogni ciclo di clock, un input appartenente all'alfabeto $I = \{\text{nessuno}, \text{primo}, \text{secondo}\}$, per indicare quale giocatore ha vinto un game e fornisce in uscita un valore appartenente all'alfabeto $O = \{\text{incorso}, \text{primo}, \text{secondo}\}$ ad indicare qual è il vincitore del set. Quando un giocatore raggiunge due game vinti, la macchina segnala il vincitore e torna allo stato iniziale per un nuovo set. Si realizzi il circuito equivalente dell'equazione di eccitazione minimizzata di un flip flop di stato a scelta.

¹: Un giocatore vince il set se vince due game prima dell'avversario. Se entrambi i giocatori vincono un game ciascuno (situazione 1-1), il vincitore del set è determinato dal terzo game.

Esercizio 3 (12 punti): Convertire in C il seguente frammento di codice assembly.

```
process:
    xorq %rdx, %rdx
    xorl %r8d, %r8d
.loop:
    cmpq %rsi, %rdx
    jz .done
    movw (%rdi, %rdx, 2), %ax
    cmpw $0, %ax
    jns .skip
    negw %ax
.skip:
    addq $1, %rdx
    movswl %ax, %eax
    addl %eax, %r8d
    jmp .loop
.done:
    mov %r8d, %eax
    ret
```

Soluzioni

Esercizio 1

Assumiamo $X = 5, Y = 6$. Il numero da convertire è quindi -55.61 .

Convertiamo per prima cosa la parte intera con divisioni successive: $55_{10} = 110111_2$.

Successivamente, convertiamo la parte frazionaria con moltiplicazioni successive:

$$0.61 \times 2 = 1.22 \quad (1)$$

$$0.22 \times 2 = 0.44 \quad (0)$$

$$0.44 \times 2 = 0.88 \quad (0)$$

$$0.88 \times 2 = 1.76 \quad (1)$$

$$0.76 \times 2 = 1.52 \quad (1)$$

$$0.52 \times 2 = 1.04 \quad (1)$$

$$0.04 \times 2 = 0.08 \quad (0)$$

$$0.08 \times 2 = 0.16 \quad (0)$$

$$0.16 \times 2 = 0.32 \quad (0)$$

$$0.32 \times 2 = 0.64 \quad (0)$$

$$0.64 \times 2 = 1.28 \quad (1)$$

$$0.28 \times 2 = 0.56 \quad (0)$$

$$0.56 \times 2 = 1.12 \quad (1)$$

$$0.12 \times 2 = 0.24 \quad (0)$$

$$0.24 \times 2 = 0.48 \quad (0)$$

$$0.48 \times 2 = 0.96 \quad (0)$$

$$0.96 \times 2 = 1.92 \quad (1)$$

$$0.92 \times 2 = 1.84 \quad (1)$$

$$0.84 \times 2 = 1.68 \quad (1)$$

eccetera

ottenendo quindi $0,61_{10} \approx 0,1001110000101000111_2$, da cui $55,61_{10} \approx 110111,1001110000101000111_2$

Spostiamo a questo punto la virgola per ottenere una forma normalizzata:

$$110111.1001110000101000111_2 = 1.101111001110000101000111_2 \times 2^5$$

Possiamo quindi codificare in formato IEEE 754:

1. **Segno:** il numero è negativo, quindi **S = 1**.
2. **Esponente:** usiamo un bias di 127: $E = 5 + 127 = 132_{10} = 10000100_2$
3. **Mantissa:** prendiamo i primi 23 bit della parte frazionaria.

Ora componiamo il numero IEEE 754 a 32 bit:

1 10000100 10111100111000010100011

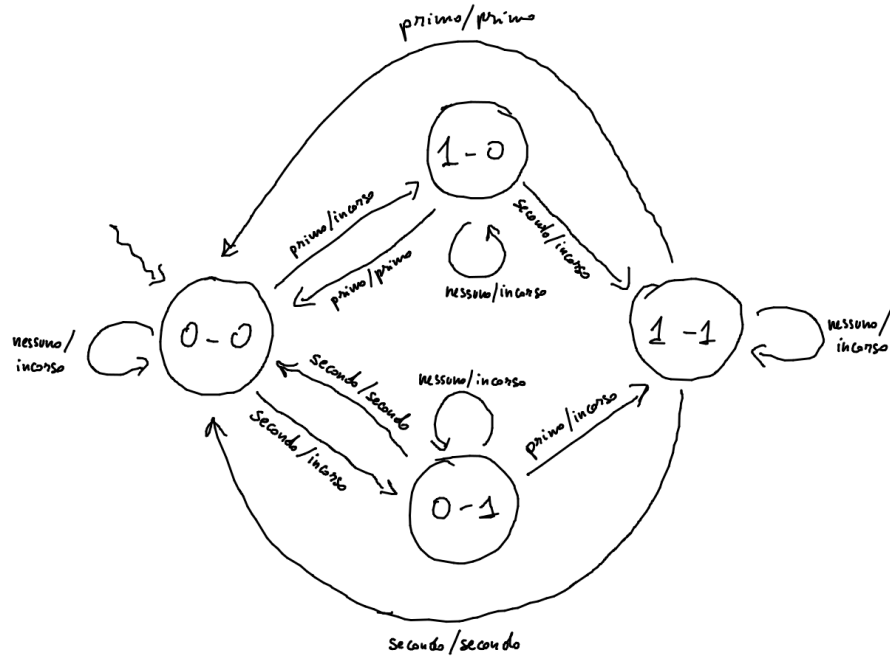
Per convertire in esadecimale, raggruppiamo a gruppi di 4 bit ed effettuiamo la conversione gruppo a gruppo:

1100 0010 0101 1110 0111 0000 1010 0011

$C25E70A3_{16}$

Esercizio 2

L'automa è essenzialmente un contatore che può essere rappresentato come segue:



Adottando le seguenti codifiche

S	y_1	y_0	I	x_1	x_0	O	z_1	z_0
0-0	0	0	nessuno	0	0	in corso	0	0
0-1	0	1	primo	0	1	primo	0	1
1-0	1	0	secondo	1	0	secondo	1	0
1-1	1	1						

si ottiene la seguente tabella degli stati e delle transizioni:

y_1	y_0	x_1	x_0	y_1'	y_0'	z_1	z_0
0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	1	0	0
0	1	0	1	1	1	0	0
0	1	1	0	0	0	1	0
1	0	0	0	1	0	0	0
1	0	0	1	0	0	0	1
1	0	1	0	1	1	0	0
1	1	0	0	1	1	0	0
1	1	0	1	0	0	0	1
1	1	1	0	0	0	1	0

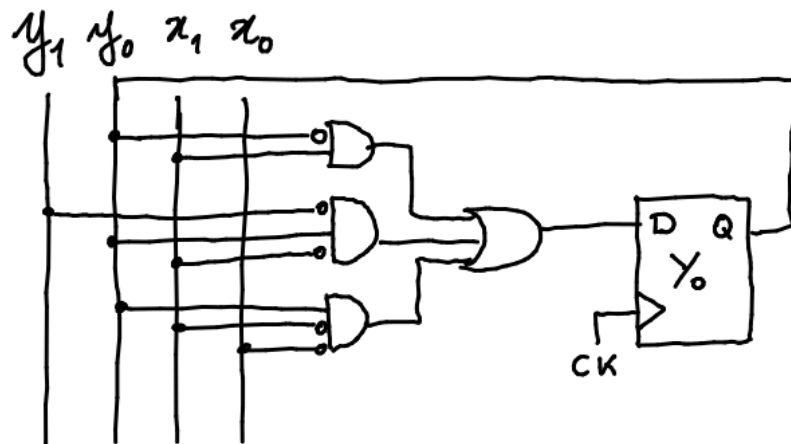
Scegliendo di realizzare l'equazione di eccitazione di y_0' , possiamo costruire la seguente mappa di Karnaugh

Handwritten Karnaugh map for variables y_1, y_0 (columns) and x_1, x_0 (rows). The map shows the following values:

	00	01	11	10
00	0	1	1	0
01	0	1	0	0
11	-	-	-	-
10	1	0	0	1

Red circles highlight the 1s in the first two rows, and red brackets highlight the 1s in the first and last columns.

da cui si ottiene l'equazione minimizzata $y'_0 = \overline{y_0}x_1 + \overline{y_1}y_0\overline{x_1} + y_0\overline{x_1}x_0$ che porta al circuito equivalente:



Esercizio 3

Il frammento di codice assembly azzerava il registro `r8d` (linea 3) che viene poi copiato in `eax` prima di eseguire la `ret` (linee 17 e 18). Si tratta quindi di una funzione che ha un valore di ritorno di dimensione 32 bit.

Alle linee 5 e 7 vengono utilizzati i registri `rdi` ed `rsi`, che non sono stati scritti precedentemente nel frammento. Quindi, per le calling convention, la funzione deve accettare due parametri, entrambi di 64 bit. Il registro `rdi` viene utilizzato come base in un accesso a memoria (linea 7), quindi deve essere un puntatore. Nell'accesso a memoria, viene utilizzato l'opcode `movw`, quindi `rdi` deve essere un puntatore a dati a 16 bit. Pertanto, la firma della funzione deve essere qualcosa del tipo:

```
int process(short *a, long long b);
```

Nell'accesso a memoria, viene utilizzato come registro indice `rdx`, che è stato precedentemente inizializzato a 0 (linea 2). Il valore viene ripetutamente confrontato con `rsi`, suggerendo quindi che il secondo parametro sia la taglia di un vettore. Poiché il primo parametro è usato come registro base, quello sarà il vettore che si vuole scandire.

Vi è un ciclo, all'interno del quale:

1. Viene confrontato l'elemento i -esimo con zero (linea 8)

2. Si verifica se il risultato della `cmp` sia minore di zero (controllando il SF). Si noti che non ci può essere overflow in questo caso, poiché si sta sottraendo zero: pertanto controllare SF è sufficiente per verificare se `ax < 0`.
3. Se $SF \neq 1$, non si fa niente, viceversa si calcola il complemento a 2 di `ax` con l'istruzione `neg`.
4. In entrambi i casi, si somma il contenuto di `ax` in `r8d` dopo aver effettuato la conversione da word a longword. Poiché non viene mai sommato un valore negativo in `r8d`, il tipo di quella variabile può essere `unsigned`.

Rispetto al punto 4 qui sopra, la conversione da un tipo a meno bit a un tipo a più bit in C è sempre implicita e definita, pertanto l'istruzione a riga 13 non comparirà nel codice C che, dato quanto scritto sopra, potrà essere il seguente:

```
unsigned process(short *array, long long count)
{
    unsigned sum = 0;
    for (long long i = 0; i < count; ++i) {
        short x = array[i];
        if (x < 0)
            x = -x;
        sum += x;
    }
    return sum;
}
```

Progetto

Un processore `z64` controlla una valvola di sicurezza in un condotto di gas ad alta pressione. Il sistema è costituito da due dispositivi: il sensore `MANOMETRO` e l'attuatore `ELETTROVALVOLA`. `MANOMETRO` è una periferica asincrona che, ogni 50 millisecondi, genera una richiesta di interruzione per segnalare la disponibilità di una nuova misurazione della pressione nella tubazione.

Alla ricezione di una richiesta di interruzione, il processore `z64` acquisisce la k -esima lettura della pressione, denotata come $P(k)$, e aggiorna una media mobile calcolata sulle quattro misurazioni più recenti mediante la formula $M(k) = \frac{1}{4} \sum_{i=0}^3 P(k-i)$.

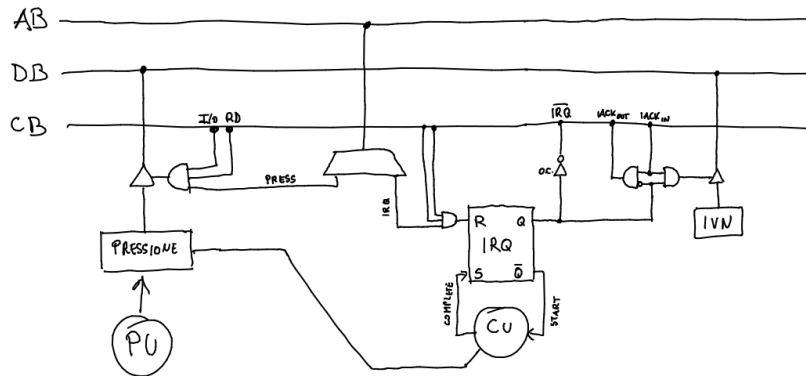
I valori di riferimento per il controllo del flusso sono memorizzati in due variabili in memoria (`above` e `below`) che definiscono una fascia di isteresi. Se la media mobile $M(k)$ supera la soglia superiore della fascia, lo `z64` comanda `ELETTROVALVOLA` affinché chiuda il flusso di gas. Se invece la media mobile scende al di sotto della soglia inferiore, lo `z64` ordina la riapertura della valvola, ripristinando il flusso di gas.

Realizzare:

- Le interfacce dei dispositivi `MANOMETRO` ed `ELETTROVALVOLA`;
- Tutto il codice necessario al funzionamento del sistema, senza l'utilizzo di pseudo-istruzioni.

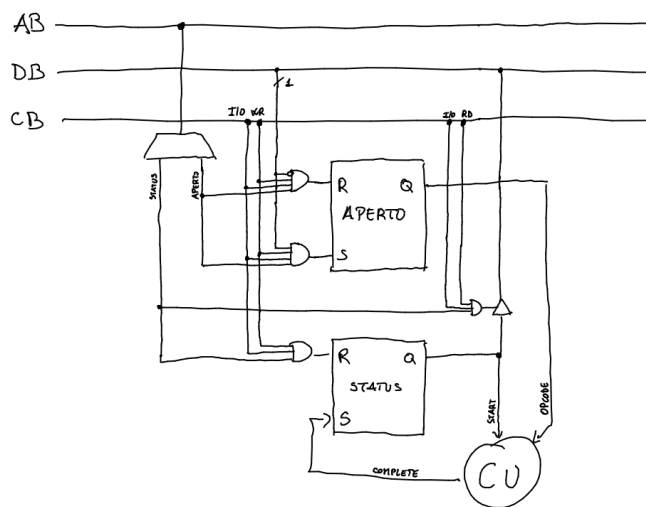
Soluzione

MANOMETRO è una periferica asincrona di input, la cui interfaccia può essere realizzata come segue:



La CU, utilizzando il valore del F/F IRQ, garantisce che non venga sovrascritto il contenuto del registro PRESSIONE prima che la CPU abbia acquisito il valore prodotto in precedenza. Il fatto che il dispositivo invii interruzioni ogni 50 millisecondi è un dettaglio interno del dispositivo di cui non ci interessa nulla in fase di interfacciamento.

ELETTROVALVOLA è una periferica sincrona, che deve essere programmabile per consentire alla CU di identificare se la CPU sta richiedendo l'apertura o la chiusura della tubazione. Si può quindi realizzare l'interfaccia come segue:



Assumendo, come è giusto che sia, che la pressione non possa essere negativa, u

na possibile implementazione del software è la seguente.

```
.org 0x800
.data
.equ PRESSIONE, 0x0
.equ MAN_IRQ, 0x1
.equ EL_APERTO, 0x2
.equ EL_STATUS, 0x3
above: .long ...
below: .long ...
misurazioni: .long 0,0,0,0
pos: .word 0
.text
main:
# Inizializza misurazioni al valor medio (above e below sono variabili)
```

```

    movl below, %eax
    addl above, %eax
    sarl $1, %eax
    movq $4, %rcx
    movq $misurazioni, %rdi
    cld
    stosl

    sti

.loop:
    call media
    cmpl above, %eax
    jc .chiudi
    cmpl %eax, below
    jnc .loop
    call apri
    jmp .loop
.chiudi:
    call chiudi
    jmp .loop

media:
    movl misurazioni, %eax
    addl misurazioni+4, %eax
    addl misurazioni+8, %eax
    addl misurazioni+12, %eax
    sarl $2, %eax
    ret

apri:
    movl $0, %eax
    jmp .attua
chiudi:
    movl $1, %eax
.attua:
    outl %eax, $EL_APERTO
    outb %al, $EL_STATUS
.bw:
    inb $EL_STATUS, %al
    btb $0, %al
    jnc .bw
    ret

.driver 0 # MANOMETRO
    push %rax
    push %rcx
    inl $PRESSIONE, %eax
    movzwq pos, %rcx
    movl %eax, misurazioni(, %rcx, 4)
    addq $1, %rcx
    andl $3, %rcx
    movw %cx, pos
    outb %al, $EL_STATUS
    pop %rcx
    pop %rax
    iret

```

